Final Project - Stat215A, Fall 2025

January 9, 2026

1 Introduction

Natural speech is complex. It contains layers of information: syntax, semantics, prosody, and even social or pragmatic cues. To derive meaning from speech, the human brain must integrate the information carried by individual words with the surrounding linguistic context. Without this contextual integration, it would be impossible to interpret the full richness and ambiguity of natural language [4].

Because this contextual information is represented in the cortex, mapping speech input onto cortical activation patterns provides a powerful way to identify which brain regions encode specific linguistic features. This approach offers important insights into how the brain processes and understands language. In this project, we address the following question: *How does the human cortex represent semantic and contextual information during natural speech perception?*

Despite substantial progress in computational linguistics, it remains an open question which kinds of linguistic representations best reflect how the human brain processes natural language. Classical distributional embeddings such as Word2Vec and GloVe capture global co-occurrence statistics, whereas transformer-based representations such as BERT encode rich contextual information that varies with each usage of a word. Determining which type of representation best predicts neural responses can shed light on the nature of cortical semantic processing and the degree to which contextual structure is reflected in brain activity.

In this project, we systematically compare four embedding models (Word2Vec, GloVe, pretrained BERT, and a BERT model fine-tuned on the story corpus) using voxel-wise ridge regression encoding models. Our results show that the fine-tuned BERT embeddings outperform all other models across major evaluation metrics, suggesting that contextualized semantic representations best align with the neural processing of narrative language in the human brain.

2 Data

We use two different datasets in this report: a text dataset and an fMRI dataset. The text dataset contains the story transcripts that were spoken to each subject, while the fMRI dataset captures the voxel-wise brain responses elicited by those stories.

2.1 Text Dataset

The first dataset consists of the raw story transcripts (`raw_text.pkl`). This dataset includes 109 stories, each containing between 697 and 3,476 words and between 278 and 1,052 unique words (Figure 1). Across all stories, the total number of words is 205,415, and the total number of unique words is 12,903, indicating substantial repetition within the corpus. For example, the word *and* appears 9,952 times, and *the* appears 8,526 times (Figure 2). Because many of the most frequent words are articles, pronouns, conjunctions, and prepositions—terms that often carry limited semantic content—it is important to ensure that the model does not overfit to these high-frequency but low-information tokens.

2.2 fMRI Experiment

The fMRI dataset used in this project was collected as part of a naturalistic language-comprehension experiment in which subjects listened to spoken narrative stories while fMRI responses of the whole brain were recorded. The stimulus consisted of naturally spoken stories taken from *The Moth Radio Hour*, totaling over two hours of audio and approximately 23,800 words across all narratives [4]. Each story was manually transcribed and aligned to the audio waveform, which allows the precise onset time of every spoken word

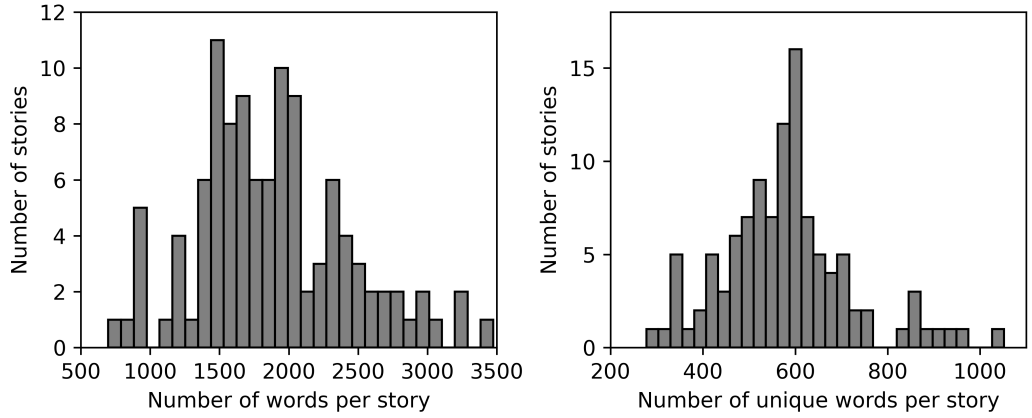


Figure 1: (left) Histogram of total words per story. (right) Histogram of unique words per story.

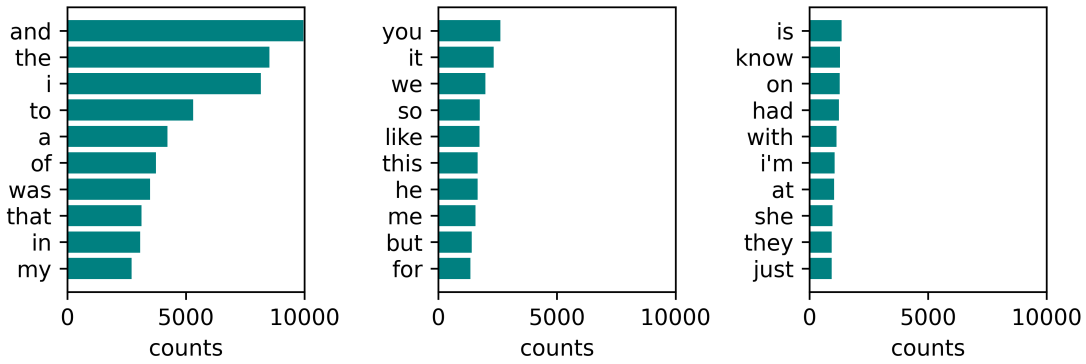


Figure 2: Top 30 most frequent words across all 109 stories.

to be identified. The time-alignment is essential for building language encoding models, because it enables direct mapping between the temporal progression of the linguistic stimulus and the corresponding voxel-wise BOLD responses.

During the experiment, subjects (six in total in the original study, two available in our dataset) listened to the stories while undergoing whole-brain fMRI scanning. BOLD data were acquired using gradient-echo echo-planar imaging (EPI) at the UC Berkeley Brain Imaging Center with TR = 2.0045 seconds, TE = 31 ms, a 3T Siemens TIM Trio scanner, and a 32-channel head coil. Each scan produced a time series of approximately 4,000 whole-brain volumes per subject, with a voxel size of $2.24 \times 2.24 \times 4.10$ mm (30 axial slices) [4]. These acquisition parameters define the temporal and spatial resolution of the neural responses used for model training. For each subject and each story, the fMRI response is stored as a matrix of size $T' \times V$, where T' is the number of fMRI measurement time points (TRs), and $V = 95{,}556$ is the number of voxels in the whole-brain recording.

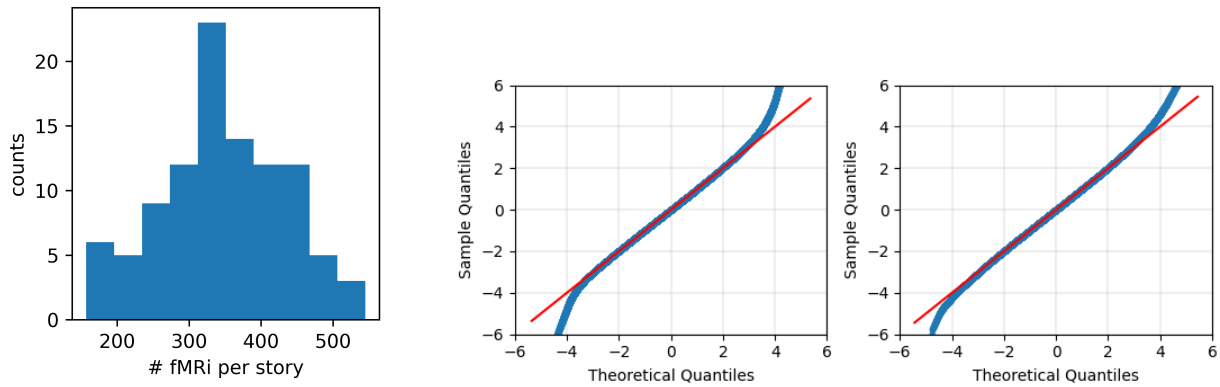
A naturalistic experimental paradigm is particularly suited for studying semantic processing because understanding narrative speech relies heavily on context, temporal dependencies, and high-level conceptual structure. Unlike isolated words or sentences, continuous stories engage a wide range of linguistic, cognitive, and conceptual processes, thereby producing rich patterns of cortical activity that are well-suited for testing different linguistic feature representations. For this reason, the dataset has been widely used to benchmark language encoding models and to study how semantic information, context, and temporal structure are represented across the cortex.

2.3 Exploratory Data Analysis

We noticed that the fMRI dataset does not include all 109 stories. The missing stories are:

`dialogue1`, `dialogue2`, `dialogue3`, `dialogue4`, `dialogue5`, `dialogue6`, `myfirstdaywiththeyankees`, `onlyonewaytofindout`.

Across the 101 available stories, the number of fMRI observations (time points) ranges from 157 to 545 TRs, with an average of approximately 344 TRs (Figure 3a). A TR (repetition time) corresponds to one whole-brain fMRI volume acquisition, so the number of TRs represents how many consecutive brain measurements were recorded for each story. Additionally, the voxel intensities for each story appear to be have been standardized prior to release, following approximately a standard normal distribution $\mathcal{N}(0, 1)$, as illustrated in the QQ plots in Figure 3b.



(a) Distribution of the number of fMRI time points

(b) Examples of subject 3's voxel-intensity QQ plots for two stories: (left) *adollshouse*, (right) *adventuresinsayingyes*.

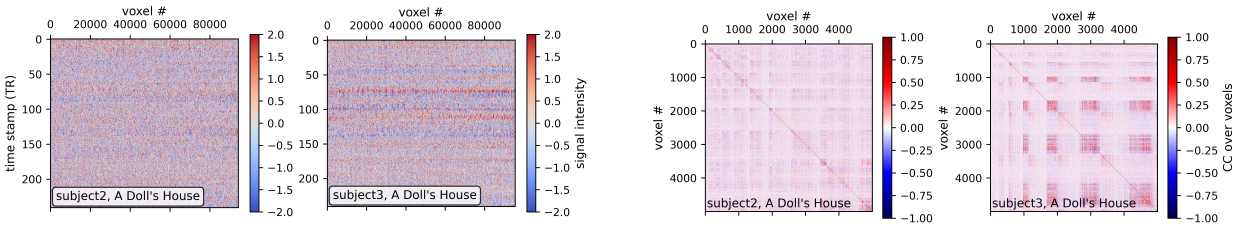
Figure 3: Data statstics on fMRI reponse data

Also, the voxel responses to the same story appear to differ across subjects, especially in their overall intensity. In Figure 4a, even for the same story, the responses of subject 2 and subject 3 are not consistent with each other. Subject 3 generally exhibits a stronger signal than subject 2. This stronger response for subject 3 can also be seen in the voxel-wise correlation coefficients (Figure 4b).

In Figure 4b, as expected, adjacent voxels tend to have similar responses. It is also interesting that the voxel-wise correlation coefficients appear to be periodic: the correlations seem to exhibit a repeating pattern of similar values across different voxels. This suggests that flattening the three-dimensional brain volume into a one-dimensional array (e.g., concatenating voxel indices along horizontal or vertical directions) induces this apparent periodicity, because voxels that are close in physical space are mapped to positions that are regularly spaced along the one-dimensional index.

2.4 Data cleaning

The data-cleaning procedure was carried out as follows. Starting from the original `raw_text.pkl` file, we first removed leading and trailing spaces from each token and discarded any tokens consisting only of whitespace. Next, we checked each token against standard English dictionaries using the *WordNet* and *stopwords* corpora from NLTK to identify words not present in the dictionary. This process produced 1,379 out-of-dictionary tokens. We manually inspected each of them to determine whether it represented a genuine misspelling or was simply absent from WordNet. We found that most tokens were legitimate but not included in the dictionary, such as people's names (e.g., *Alyssa*, *Betty*), brand names (e.g., *NBC*, *McDonald's*), expressions of disfluency (e.g., *duh*), contractions (e.g., *how'd*, *I've*), or possessive forms (e.g., *child's*). Among the 1,379 tokens, 41 were identified as clear typographical errors and 11 as non-words. The misspellings and their corrections are listed in Table 1, and the eleven non-words are:



(a) Voxel response intensity.

(b) Voxel–voxel correlation matrix.

Figure 4: Overview of voxel-level fMRI responses for the story "A Doll's House." (subject 2 left, subject 3 right).

```
's, (br}, (br}', \andm, {cg}, {ig}, {ls), {ls} , {ns], {sl}, f{ns}
```

We corrected the 41 typographical errors according to Table 1 and removed the eleven non-words. As a result, the cleaned dataset contains a total of 205,246 words with 12,846 unique words, preserving more than 99% of the original tokens (originally 205,415 tokens with 12,903 unique words).

Table 1: List of typos and their corrections

Typo	Correct	Typo	Correct	Typo	Correct	Typo	Correct
tennant	tenant	probablys	probably	fiften	fifteen	and'	and
secre	secret	intp	into	weaknesst	weakest	happene	happen
deppressing	depressing	whatw	what	that"s	that's	trow	throw
so]]	so	thikn	think	sid	side	bu	but
ofso	of	rea	real	walkd	walked	complulsively	compulsively
undertand	understand	welcone	welcome	abercormbie	abercrombie	temping	tempting
absinence	abstinence	adopteest	adoptees	inactivtiy	inactivity	onw	one
happeist	happiest	saety	safety	botter	bottle	chekcpoint	checkpoint
ao1	all	thity	thirty	successfull	successful	chaplainsometimes	chaplain
wasw	was	spanding	spanning	starte	started	did'nt	didn't
thier	their						

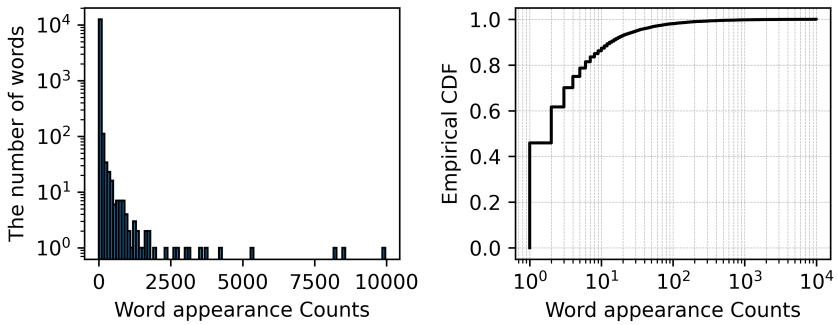


Figure 5: (left) the histogram of the word appearance count and (right) its cumulative distribution function.

3 Word Embeddings

3.1 Word2Vec

Word2Vec is a class of neural word embedding models that learn distributed vector representations of words based on their distributional properties in large text corpora. The core idea is that words appearing in

similar linguistic contexts tend to have similar meanings. Word2Vec operationalizes this idea using a shallow neural network trained with one of two architectures: continuous bag-of-words (CBOW), which predicts a target word from its surrounding context, and skip-gram, which predicts surrounding context words from a target word. Through this predictive objective, the model learns low-dimensional vector embeddings in which semantic and syntactic relationships between words are captured as geometric relationships in the embedding space. Importantly, Word2Vec produces static embeddings—each word is assigned a single vector regardless of context.

Word2Vec in our implementation operates on whole words rather than subword tokens. Our objective is not to capture the grammatical properties of words, but rather their contextual meaning. For example, purely morphological variations such as past tense forms should not substantially change the vector representation of a word. The hyperparameters of the model are selected considering this purpose.

Hyperparameters Word2Vec has several primary hyperparameters: `vector_size`, `window_size`, `min_count`, `sg`, `hs`, `negative`, `epochs`, `alpha`, `sample`, and `workers`. The `vector_size` parameter defines the dimensionality of the word embeddings. If the vector size is too large, the model may overfit the data, whereas too small a size may lead to poor representations. In our experiments, we found that increasing the vector size beyond 300 did not substantially affect the resulting word vectors. Therefore, we selected a vector size of 300 in this report to minimize the computational burden.

The `window_size` parameter defines the span of context around a target word. For a window size of c, Word2Vec uses the c words before and after the target word, i.e., up to $2c + 1$ words in total, to learn its representation. Given our objective of predicting brain responses when a word is heard, it is essential that the embeddings capture the contextual meaning of the word rather than only grammatical or structural similarity. We selected $c = 10$ (i.e., a context of 21 words), considering that an English sentence is on average about 20–25 words long [2].

The `min_count` parameter specifies the minimum number of occurrences a word must have in the corpus to be included in training and assigned an embedding. If `min_count` $= k$, words appearing fewer than k times are ignored, and only words appearing at least k times are represented in the output. We set `min_count` $= 1$ so as not to exclude potentially meaningful words.

In addition, words that are meaningful for fMRI activation tend to be relatively infrequent, whereas highly frequent words such as *I*, *uh*, etc. carry little semantic content for our purposes. For this reason, we used hierarchical softmax (`hs` $= 1$), which generally performs well for rare words.

The `sg` parameter specifies whether to use the CBOW (Continuous Bag-of-Words) model or the skip-gram model, which operate in opposite directions. CBOW predicts the target word from its surrounding context words, while skip-gram predicts the context words from the target word. Skip-gram is computationally slower than CBOW, but tends to perform better on rare words and still works well for frequent words [5]. We chose skip-gram (`sg` $= 1$) because it generally works better for smaller corpora and less frequent words. In our case, the corpus consists of approximately 200K words with 12,846 unique words, and more than 80% of the words appear fewer than 10 times (Figure 5).

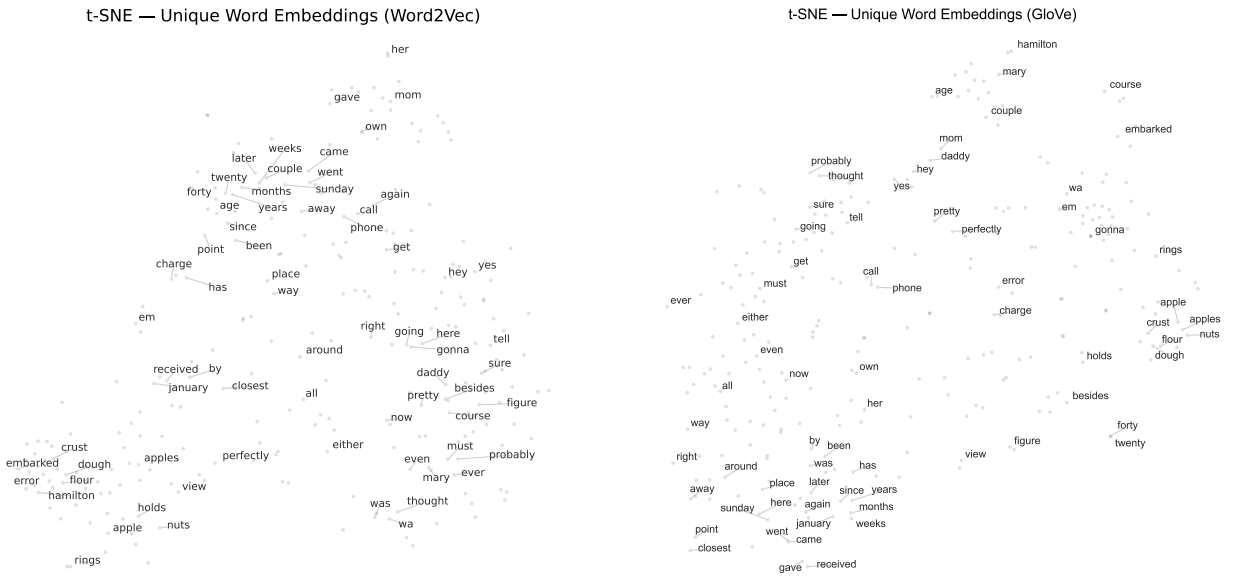
Downsampling of frequent words is also important in this project, to avoid overfitting to overly common but semantically weak words such as *I*, *she*, etc. We used the `sample` parameter of 10^{-3}, which is the default value suggested in Gensim's Word2Vec implementation, and also provides reasonable relationship between words (Figure 6a). Although the typically recommended range goes down to 10^{-5}, we found that smaller `sample` does not preserve the word's relationship as a vector.

In addition, we did not use pre-trained embeddings such as Word2Vec Google News because the text we analyze differs from the formal, edited language found in news articles. Table 2 summarizes the hyperparameters selected for this project.

Visualization To inspect the structure of the learned embedding space, we averaged multiple occurrences of each unique token and visualized a two-dimensional representation using PCA (50 components) followed by t-SNE. We do this consistently for each of our four generated embeddings (Word2Vec, Glove, pre-trained

Table 2: Word2Vec embeddings hyperparameter

name	value	name	value	name	value
vector_size	300	window_size	10	min_count	1
sg	1 (skip-gram)	hs	1 (Hierarchical softmax)	downsampling	10^{-3}



(a) Word2Vec

(b) GloVe

Figure 6: t-SNE visualizations of Word2Vec and GloVe embeddings for the most frequent unique words.

BERT, and fine-tuned BERT). Figure 6a visualizes our Word2Vec embeddings. Overall, the embedding space shows meaningful clustering of semantically related words such as the words associated with numbers and time or food-related words. The discussion on the word embeddings are extensively discussed in section 3.6

3.2 GloVe

In addition to the previously discussed Word2Vec embedding, we used GloVe (Global Vectors for Word Representation) [6]. GloVe is built on the idea that the meaning of a word can be inferred from the context around it: words that appear in similar contexts across a large text corpus should receive similar vector representations. Unlike predictive models that directly try to guess the next word, GloVe is a more *global* method that uses statistics from the entire corpus to learn embeddings.

Formally, GloVe begins by constructing a large word-word co-occurrence matrix X, where X_{ij} counts how often word j appears in the context of word i. Mathematically, GloVe minimizes the weighted least-squares objective

$$J = \sum_{i,j} f(X_{ij}) \left(w_i^\top \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2,$$

where w_i and $\tilde{w}_j$ are the target and context embeddings, b_i and $\tilde{b}_j$ are bias terms, and $f(X_{ij})$ is a weighting function that limits the influence of very frequent pairs. This links geometric distances in the embedding space with statistical structure in the corpus: if two words have similar co-occurrence patterns (e.g., “king” and “queen”), their vectors will lie close together.

A key intuition behind GloVe is that certain semantic relations can already be detected in simple ratios of co-occurrence probabilities. Consider two target words, such as *ice* and *steam*, and a set of probe words

k. Let $P(k \mid i)$ denote the probability that k appears in the context of word i. Words that are strongly associated with *ice* (such as *solid*) yield ratios $P(k \mid \text{ice})/P(k \mid \text{steam}) \gg 1$, whereas words associated with *steam* (such as *gas*) yield ratios $\ll 1$. Words that are equally unrelated to both (e.g., *fashion*) have ratios close to 1. These ratios capture a crude notion of meaning because they highlight discriminative contextual information while canceling noise from non-informative co-occurrences. Table 3 shows some real probabilities from a 6-billion-token corpus reported in the GloVe paper [6].

Table 3: Co-occurrence probabilities and ratios for the target words *ice* and *steam*.

Probability and Ratio	$k = \text{solid}$	$k = \text{gas}$	$k = \text{water}$	$k = \text{fashion}$
$P(k \mid \text{ice})$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k \mid \text{steam})$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(k \mid \text{ice})/P(k \mid \text{steam})$	8.9	8.5×10^{-2}	1.36	0.96

These ratios motivate the GloVe objective: because $\log\left(\frac{P_{ik}}{P_{jk}}\right) = \log P_{ik} - \log P_{jk}$, learning vectors whose dot products approximate $\log X_{ij}$ naturally encodes such ratios as vector differences. As a result, meaningful semantic dimensions emerge in the embedding space, giving GloVe a good performance on analogy tasks.

Compared to Word2Vec, which learns embeddings by predicting a word from its context (Skip-Gram) or predicting the context from a word (CBOW), GloVe does not use a local prediction task but instead factorizes the global co-occurrence structure of the entire corpus. As a result, GloVe often captures more stable global semantic relationships. On the other hand, BERT embeddings, cf. section 3.3 and 3.4, come from a deep transformer model and are contextual: the vector for a word changes depending on the sentence in which it appears. GloVe (and Word2Vec), however, produce static embeddings: each word has one fixed vector regardless of context. This simplicity makes GloVe computationally efficient and therefore useful for fMRI encoding studies, where a consistent representation per word is desirable.

For our project, we used the pretrained `glove.2024.wikigiga.300d` model, which was trained on a corpus consisting of the combined English Wikipedia dump and Gigaword 5 (11.9 billion tokens). The model is uncased, uses a vocabulary of approximately 1.2 million tokens, and provides 300-dimensional vectors. We embedded each word via direct dictionary lookup, assigning zero vectors to out-of-vocabulary tokens as a conservative strategy to maintain alignment between word timing and embedding sequences.

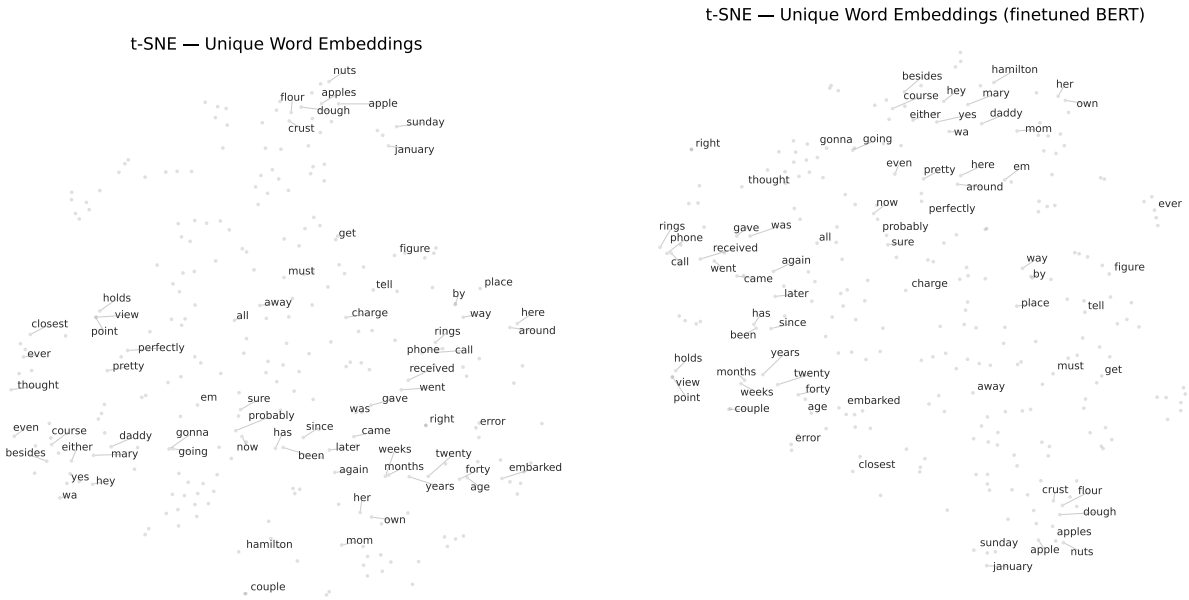
The t-SNE visualization in Figure 6b shows the 300-dimensional GloVe word embeddings projected into two dimensions. It shows how semantically related words cluster together in the embedding space. We observe meaningful groupings such as food-related words (crust, nuts, apples) on the right of the plot, time (months, weeks, years, etc.) on the lower left, and relationships (couple, mom, daddy) at the top.

3.3 Pretrained BERT

We computed embeddings using the bert-base-uncased model from the HuggingFace `transformers` library. BERT (Bidirectional Encoder Representations from Transformers) is a large language model trained using a masked language modeling objective on a large corpus of English text. Crucially, BERT builds representations that incorporate bidirectional context: for each token, the model reflects information from both preceding and succeeding words in a sentence rather than relying solely on co-occurrence statistics as in classical distributional methods such as Word2Vec or GloVe.

Before passing text to BERT, we used the `BertTokenizerFast` tokenizer to segment raw word sequences into subword units. The tokenizer performs lower-casing, punctuation splitting, and Byte-Pair Encoding (BPE), which allows the model to robustly handle rare or out-of-vocabulary terms by decomposing them into subwords instead of discarding them. This step is essential because the raw transcripts contain conversational and narrative language with variable morphology, names, and multi-token expressions (e.g., “thanksgiving,” “hamilton,” and “apple dough”).

Because BERT operates on subword tokens, we aggregated hidden states back to the word level. For a sequence of T words, let $h_{i,j} \in \mathbb{R}^{768}$ denote the hidden state for the j-th subword associated with word i.



(a) Pretrained BERT embeddings. The t-SNE visualization shows broadly meaningful groupings (e.g., food words such as "apples", "flour", "dough", "nuts" appear near each other), but clusters are relatively diffuse with less separation between semantic themes.

(b) Finetuned BERT embeddings. Compared with the pretrained space, the finetuned model produces noticeably tighter and more coherent clusters (e.g., clearer separation of food-related terms and social/family-related terms), indicating that finetuning sharpened semantic structure in the embedding space.

Figure 7: Side-by-side comparison of t-SNE embeddings for averaged unique-word representations using pretrained (left) and finetuned (right) BERT models. The finetuned space exhibits tighter semantic neighborhoods and clearer thematic structure than the pretrained version, consistent with improved downstream encoding performance.

For each word, we averaged these subword vectors:

$$\mathbf{e}_i \;=\; \frac{1}{n_i}\sum_{j=1}^{n_i} h_{i,j}\,,$$

where n_i is the number of BERT subwords corresponding to word i. This yields a matrix of embeddings $\mathbf{E} \in \mathbb{R}^{T\times 768}$ aligned with stimulus word timestamps. To avoid exceeding BERT's maximum sequence length, we chunked each story into segments of 256 words and processed them independently on GPU.

As shown in Figure 7a, semantically related words form clusters, including thematic groups related to seasonal events ("thanksgiving," "sunday," "january"), family members ("mom," "father," "daddy"), and food preparation ("flour," "crust," "apples," "dough"). This suggests that even without task-specific fine-tuning, BERT captures rich lexical and semantic structure present in the narrative stimuli. The visualization uses the top 70 tokens selected by the ℓ_2 norm of their embeddings. The clear structure in Figure 7a reinforces the motivation for including BERT-based embeddings in language encoding models. Relative to static embeddings, BERT provides contextualized representations, which prior work has shown to significantly improve encoding model performance on fMRI language tasks.

3.4 Finetuned BERT

To obtain task-specific semantic embeddings aligned with the fMRI decoding task, we fine-tuned a `bert-base-uncased` model using the masked-language-modeling (MLM) objective on the cleaned story transcripts. Because our dataset is small compared to BERT's pretraining corpus, we adopted the Low-Rank Adaptation (LoRA) method from [3], which enables efficient fine-tuning by updating only a low-dimensional subspace of the model's weight matrices. This approach reduces memory usage, prevents overfitting, and fits well within the

GPU and disk constraints on PSC Bridges-2.

The resulting encoder was used to extract 768-dimensional contextual embeddings for every word in every story.

LoRA LoRA modifies selected linear layers within a pretrained transformer by introducing trainable low-rank matrices. Instead of updating the full weight matrix $W \in \mathbb{R}^{d\times d}$ during fine-tuning, LoRA parameterizes the update as

$$\Delta W = BA, \qquad B \in \mathbb{R}^{d\times r},\ A \in \mathbb{R}^{r\times d}, \quad r \ll d.$$

The effective weight used during training is then

$$W_{\text{eff}} = W + \alpha\, BA,$$

where α is a scaling factor. In our implementation, we used a rank of $r = 8$ and scaling $\alpha = 16$, with dropout $p = 0.1$ inside the LoRA branch to avoid degenerate solutions.

LoRA was applied to the self-attention query and value projection matrices in all 12 layers of BERT. This choice preserves the original pretrained features while allowing small, structured updates that adapt the model to narrative language.

Data preparation and tokenization The story texts were cleaned using a custom preprocessing script and tokenized using the `BertTokenizerFast`. Since many story transcripts exceed BERT's maximum input length, we partitioned each story into fixed-length windows of 128 tokens, padded to a consistent sequence length. A chunk size of 128 represents a compromise: smaller windows may reduce long-range semantic context, while larger windows would increase memory use.

MLM masking We use the standard BERT-style MLM objective. For each token in an input sequence, with probability 0.15 we apply one of three transformations:

- 80% of the time: replace the token with `[MASK]`,
- 10% of the time: replace it with a random vocabulary token,
- 10% of the time: leave it unchanged.

This follows the masking recipe from Devlin et al. (2019) [1].

Training We fine-tuned the model for 5 epochs using the AdamW optimizer with a learning rate of $5\cdot 10^{-5}$ and batch size 16. A validation split of 20% was created at the *story level* (not the chunk level) to prevent text from the same narrative appearing in both training and validation.

Only the MLM head and the LoRA-injected layers were updated. The remaining BERT parameters stayed frozen. This limits overfitting. A few judgement calls we made in the training configuration include:

Word-level embedding extraction After fine-tuning, we discarded the MLM head and loaded the encoder with output hidden states enabled. To obtain a contextual embedding for each word:

1. We tokenized the list of words using `is_split_into_words = true`, so each word maps to one or more subword tokens.
2. We used the mapping `word_ids()` to associate each subword with its originating word.
3. For a word consisting of subwords $t_1, \ldots, t_k$, we defined its embedding as

$$e_{\text{word}} = \frac{1}{k}\sum_{i=1}^{k} h_{t_i},$$

   where h_{t_i} is the final-layer hidden state of subword t_i.
4. Computation was performed in batches of 256 words.

This yields a $T \times 768$ embedding matrix for each story, where T is the number of words.

A quick qualitative comparison of the pretrained and finetuned BERT embedding spaces reveals some differences in their semantic organization. As shown in the t-SNE visualizations (Figure 7b), the pretrained embeddings form relatively loose and partially overlapping clusters. In contrast, the finetuned embeddings show tighter and more coherent semantic neighborhoods: food-related words form a compact cluster, relational terms group more distinctly, and verbs and functional items show reduced scatter. This suggests that finetuning successfully adapts the embedding geometry to the narrative context of the stimuli, which is consistent with the improved voxel-wise prediction performance observed in our encoding models (see Section 4.2).

3.5 Comparison on embeddings from different models

Different embeddings yield different vector representations of words. In this section, we compare how they differ and analyze the embeddings obtained from four models: Word2Vec, GloVe, pre-trained BERT, and fine-tuned BERT.

For all embeddings, the words are generally well organized into groups with similar meanings, indicating that, overall, they are well represented as vectors (Figure 6a, 6b, 7a, and 7b). For example, food-related words such as *crust*, *dough*, *flour* form clear clusters in all embeddings. Similarly, words associated with numbers and time (e.g., *forty*, *twenty*, *later*, *weeks*, *Sunday*) also appear close to one another.However, interestingly, several words reveal distinct differences among the embeddings. For example, the words *daddy* and *mom* are far apart in Word2Vec and pre-trained BERT, whereas they are close in GloVe and fine-tuned BERT.

From a contextual perspective, we find that BERT generally performs better than Word2Vec or GloVe. For the words *must* and *probably*, their relative positions change markedly across embeddings: they are very close in Word2Vec, farther apart in GloVe, and even more separated in both BERT embeddings. Note that *must* and *probably* may play similar grammatical roles as modal verbs, in that they modify how actions are expressed, but they differ substantially in meaning: *must* conveys a strong sense of obligation, whereas *probably* expresses weaker, more tentative modality. From a contextual perspective, these two words should be far apart, and this distinction is more clearly reflected in the BERT embeddings. Another example is the word *rings*, which appears close to *phone* or *call* in the BERT embeddings but does not show a similar proximity in the Word2Vec and GloVe embeddings, suggesting that the latter do not fully capture this contextual similarity.

These mismatches illustrate that the embeddings are not perfect and that some embeddings (in particular, BERT-based ones) may better capture contextual meaning. They also imply that some local neighborhoods in the two-dimensional t-SNE projection may not faithfully reflect the true semantic or contextual relationships present in the original high-dimensional embedding space.

3.6 Summary

In summary the four embedding methods illustrate a progression from purely distributional, static representations toward increasingly contextual and semantically expressive models. Word2Vec and GloVe derive meaning from word co-occurrence patterns, with Word2Vec learning local predictive relationships within a fixed window and GloVe capturing global corpus-wide statistics through matrix factorization. Their t-SNE visualizations show similar clusters that reflect sentence-independent lexical semantics, cf. Figure 6a and 6b. The pretrained and finetuned BERT models produce contextual embeddings, where a word's vector depends on its surrounding sentence and discourse. This allows them to encode subtler semantic distinctions and nuanced narrative structure that static embeddings cannot capture. The qualitative improvements seen in the finetuned BERT embedding such as tighter thematic groupings and clearer semantic neighborhoods suggest that incorporating contextual and task-adapted information may provide better representations for modeling neural responses to naturalistic language.

3.7 Data Preprocessing

Although different embedding models produce different feature representations, the raw outputs from any embedding model are not immediately compatible with the voxel-level fMRI measurements. In particular, there are three structural mismatches that must be resolved before regression can be applied: *(i) temporal*

resolution, (ii) dimensionality, and (iii) hemodynamic delay. Therefore, we applied a general preprocessing pipeline.

Temporal downsampling. Semantic embeddings are originally computed at the granularity of individual words or tokens, which results in a high-resolution time series that does not match the sampling rate of the fMRI measurements (one voxel measurement per TR). To produce a temporally consistent feature representation, we downsampled the semantic time series from word-level timestamps to fMRI TR timestamps using a continuous low-pass interpolation method. Specifically, we employed a Lanczos-windowed sinc interpolation. For each story, we consider semantic feature vectors $x(t)$ at original time points corresponding to word onsets, and we interpolate them to new target timepoints corresponding to TR boundaries. Given old timestamps $\{t_i\}$ and new timestamps $\{\tau_j\}$, the method constructs an interpolation matrix S where

$$S_{j,i} = \text{Lanczos}\big(\tau_j - t_i\big),$$

with a finite-time Lanczos window applied to the sinc kernel:

$$\text{Lanczos}(u) = \frac{\sin(\pi u)\sin(\pi u/a)}{\pi^2 u^2},$$

for a fixed window parameter a (we used $a = 3$ in our experiments). The downsampled semantic matrix is then given by

$$X_{\text{TR}} = SX_{\text{word}},$$

where X_{word} is the original matrix of semantic vectors. This interpolation procedure performs a principled low-pass filtering that prevents aliasing and preserves temporal continuity in the semantic features, rather than simply averaging or selecting nearest neighbors.

Trimming After downsampling and alignment, the semantic time series and the voxel-based BOLD measurements are nominally synchronized, but in practice their onset and offset boundaries may not correspond perfectly. The first several TRs often contain transient scanner noise and incomplete hemodynamic responses, while the final TRs may include silence or unrelated content after the story ends. To improve temporal consistency, we performed a fixed trimming step that removes the first 10 TRs and the last 5 TRs of each aligned sequence. If $X_{\text{TR}} \in \mathbb{R}^{T\times D}$ denotes the downsampled semantic matrix and $Y \in \mathbb{R}^{T\times V}$ denotes the voxel response matrix, then we construct trimmed matrices:

$$X_{\text{trim}} = X_{\text{TR}}[10 : T-5], \qquad Y_{\text{trim}} = Y[10 : T-5].$$

The trimming procedure standardizes sequence boundaries across stories and subjects, ensuring that each semantic feature vector corresponds to a well-formed neural response. Importantly, this step removes boundary artifacts without altering the internal temporal relationships needed for downstream delayed feature construction and regression.

Delayed feature construction. To model the temporal lag between stimulus presentation and neural BOLD responses, we constructed delayed versions of the semantic feature sequence. Given a semantic time series

$$X = \big[x(1), x(2), \ldots, x(T)\big] \in \mathbb{R}^{T\times D},$$

and a set of delays (measured in TRs),

$$\mathcal{D} = \{d_1, d_2, \ldots, d_k\},$$

the function constructs shifted copies of the stimulus features:

$$X^{(d)}(t) = \begin{cases} x(t-d), & t-d \geq 1, \\ 0, & \text{otherwise}, \end{cases}$$

where zero-padding is used at the boundaries. The final delayed design matrix is obtained by concatenating all shifted copies along the feature dimension:

$$X_{\text{delayed}}(t) = \left[X^{(d_1)}(t),\, X^{(d_2)}(t),\, \ldots,\, X^{(d_k)}(t)\right] \in \mathbb{R}^{T\times(D\cdot k)}.$$

In practice, we used $\mathcal{D} = \{1, 2, 3, 4\}$, which means that for each TR we include semantic features from the previous four timepoints. This approach captures the hemodynamic lag structure present in fMRI data without requiring an explicit model of the hemodynamic response function (HRF). Delayed feature construction provides a simple, non-parametric method for incorporating temporal dependencies between semantic stimuli and voxel responses.

4 Model

4.1 Implementation of Ridge Regression

We fit exact ridge regression encoding models from our four embeddings to voxelwise BOLD responses, under memory constraints. Instead of constructing the full design matrix X and response matrix Y in memory, which would be prohibitively large for high-dimensional embeddings, we designed a two-pass streaming algorithm that computes the sufficient statistics for ridge regression without ever loading all data at once.

Train/test Split We chose to perform the train/test split at the story level rather than at the scan timepoint (TR) level. Story-level splitting prevents leakage due to strong within-story temporal correlations and ensures that no TR from a given story appears in both training and test sets. Moreover, this strategy enables independent streaming over each story file, eliminating the need to concatenate TRs into a single large matrix. Here, we chose to perform an 80/20 train-test split.

Pass 1: Streaming Computation of Normalization Statistics Ridge regression requires consistent feature and response normalization. Instead of loading the full design matrix X and response matrix Y, we make a first streaming pass through all training stories to compute the feature-wise and voxel-wise means and variances:

$$\mu_X = \frac{1}{N}\sum_{t=1}^{N} X_t, \qquad \mu_Y = \frac{1}{N}\sum_{t=1}^{N} Y_t,$$

$$\sigma_X^2 = \frac{1}{N}\sum_{t=1}^{N} X_t^2 - \mu_X^2, \qquad \sigma_Y^2 = \frac{1}{N}\sum_{t=1}^{N} Y_t^2 - \mu_Y^2.$$

Certain embedding dimensions or voxels exhibit extremely small variance. To prevent division instabilities during z-scoring, we apply a `safe_std` clipping procedure:

$$\sigma_X = \max(\sigma_X, 10^{-6}), \qquad \sigma_Y = \max(\sigma_Y, 10^{-6}).$$

Pass 2: Streaming Accumulation of $X^\top X$ and $X^\top Y$ The second streaming pass accumulates the sufficient statistics for exact ridge regression. After z-scoring each chunk using the statistics computed in Pass 1, we update:

$$X^\top X \;\leftarrow\; X^\top X + X_z^\top X_z, \qquad X^\top Y \;\leftarrow\; X^\top Y + X_z^\top Y_z,$$

where

$$X_z = \frac{X - \mu_X}{\sigma_X}, \qquad Y_z = \frac{Y - \mu_Y}{\sigma_Y}.$$

Thus, the memory cost of training is dominated by storing the matrices $D \times D$ and $D \times V$, not by the number of TRs. This approach is mathematically exact and avoids the approximations typically required for mini-batch or online optimization.

Stable Solution via Eigendecomposition To estimate the ridge weights W, we solve:

$$(X^\top X + \alpha I)W = X^\top Y.$$

Rather than forming an explicit inverse, we perform a numerically stable eigendecomposition of $X^\top X$:

$$X^\top X = Q\Lambda Q^\top,$$

$$W = Q\,\mathrm{diag}\left(\frac{1}{\lambda_i + \alpha}\right) Q^\top (X^\top Y).$$

Regularization ensures that even very small eigenvalues are well-conditioned. We additionally verify that the resulting weight matrix contains no non-finite entries.

Handling Missing and Invalid Values Both embedding vectors and BOLD data occasionally contain `NaN` or $\pm\infty$ values. This would negatively impact the accumulated matrices $X^\top X$ and $X^\top Y$, so we set missing or infinite entries to zero. Since we later z-score with training means, this imputation has negligible effect and avoids dropping TRs, which would disrupt alignment between X and Y.

Cross-Validation To select the regularization parameter α, we use K-fold cross-validation at the story level. For each fold, we repeat Pass 1 and Pass 2 using only the training folds. Evaluation is performed in a streaming fashion identical to testing.

Streaming Evaluation Metrics Both MSE and voxelwise correlations are computed incrementally without storing predictions or responses:

$$\mathrm{MSE} = \frac{1}{N}\sum_{t=1}^{N}(\widehat{Y}_t - Y_t)^2.$$

Correlation is computed from accumulated means, variances, and covariances.

4.2 Model cross-validation

To evaluate the predictive performance of different semantic embedding models for voxel-wise BOLD responses, we conducted a systematic cross-validation analysis across the two subjects. For each embedding method, we trained a streaming ridge regression model using an identical 80/20 story-level split, identical hyperparameters, and identical preprocessing for all subjects. We note that Word2Vec and GloVe produce fixed 300-dimensional static embeddings, whereas BERT-based models produce 768-dimensional contextual embeddings per token (as determined by the pretrained model architecture), resulting in higher-dimensional stimulus representations.

Evaluation Metrics For each model we compute voxel-wise test correlations and summarize them using both central and tail statistics:

- `mean_cc`: mean voxel-wise correlation on the test set,
- `median_cc`: median voxel-wise correlation,
- `p95_cc`: 95th percentile (top 5% of voxels),
- `p99_cc`: 99th percentile (top 1% of voxels),
- `max_cc`: maximum voxel correlation,
- `frac_cc_gt_`τ: fraction of voxels whose correlation exceeds thresholds $\tau \in \{0, 0.05, 0.1\}$,
- `test_mse`: mean squared error on held-out stories.

Results Across both subjects, `BERT-finetuned` consistently achieves the highest performance on all major metrics, including mean correlation, median correlation, and both tail metrics ($p95$, $p99$) (Table 4 and Figure 8b). In fact, `BERT-finetuned` improves not only the "best" voxels but the overall distribution of correlations.

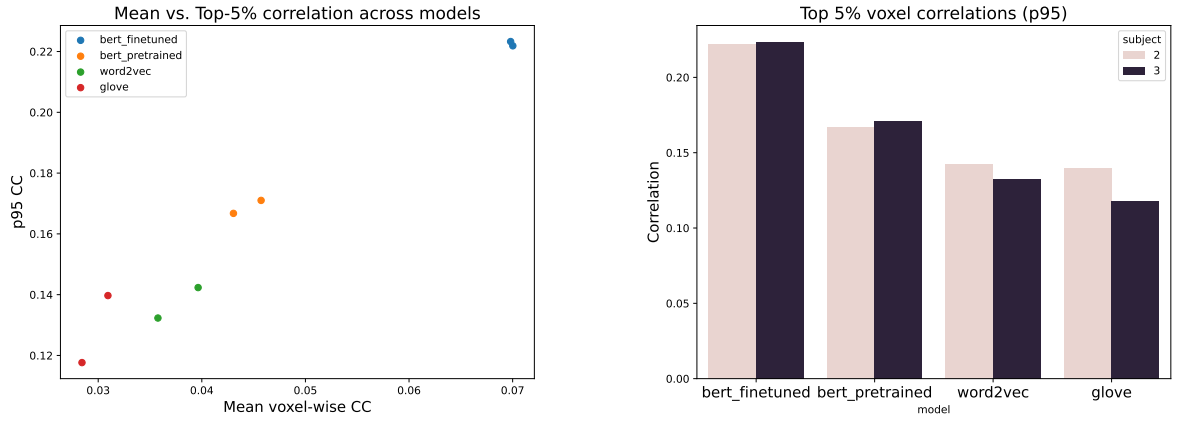
Table 4: Voxel-wise test performance across embedding models for two subjects.

(a) Subject 2

Model	MSE	mean	med	p95	p99	max	>0	>0.05	>0.1	W dims
BERT-ft	96060.64	0.0700	0.0423	0.2219	0.2977	0.4876	0.9361	0.4530	0.2642	(3072, 94251)
BERT-pre	104111.40	0.0431	0.0223	0.1667	0.2495	0.4631	0.8543	0.2877	0.1420	(3072, 94251)
Word2Vec	96669.86	0.0396	0.0236	0.1423	0.2051	0.3684	0.8546	0.2892	0.1193	(1200, 94251)
GloVe	96872.07	0.0309	0.0201	0.1397	0.2184	0.4200	0.7101	0.2616	0.1007	(1200, 94251)

(b) Subject 3

Model	MSE	mean	med	p95	p99	max	>0	>0.05	>0.1	W dims
BERT-ft	92901.92	0.0698	0.0442	0.2233	0.3135	0.5311	0.9486	0.4594	0.2442	(3072, 95556)
BERT-pre	100919.34	0.0457	0.0255	0.1710	0.2681	0.5055	0.8819	0.3000	0.1405	(3072, 95556)
Word2Vec	98106.47	0.0358	0.0217	0.1323	0.1998	0.3930	0.8468	0.2514	0.0971	(1200, 95556)
GloVe	98005.47	0.0284	0.0211	0.1177	0.1787	0.3622	0.7262	0.2525	0.0767	(1200, 95556)



(a) Mean vs. p95 correlation by model.

(b) p95 correlation for each model and subject.

Figure 8: Comparison of voxel-wise encoding performance across models.

Both the central tendency (mean and median) and the high-performance tail shift upward (Figure 8a). So, we choose `BERT-finetuned` semantic embedding model for all subsequent analyses. It consistently yields the best performance across all our computed metrics.

4.3 Model evaluation

The best-performing regression model is the fine-tuned BERT, which achieves a correlation coefficient (CC) of 0.49 for subject 2 and 0.53 for subject 3 (Table 4). For this best model selected via cross-validation, the voxel-wise CC values are plotted in Figure 9. For both subjects 2 and 3, we observe a clear periodic structure in CC across voxels. This may be because we flattened the three-dimensional cortical geometry into a one-dimensional voxel index. The periodically appearing high-correlation regions may correspond to spatially adjacent regions in physical space that control brain responses to language-related activation, such as Broca's area and Wernicke's area [7].

The Fourier amplitude spectrum of this signal provides additional insight (bottom panel of Figure 9): for both subjects, the shape of the amplitude spectrum is highly similar. In particular, both show the strong periodicity at approximately 1400 and 2700. This highlights that, regardless of the subject, the most responsive language-related regions are located at similar positions along the voxel index, even though we cannot identify their exact physical locations because we do not have a lookup table mapping voxels to anatomical coordinates. This also indicates that the regression model is robust, since the two different subjects yield

similar patterns in CC.

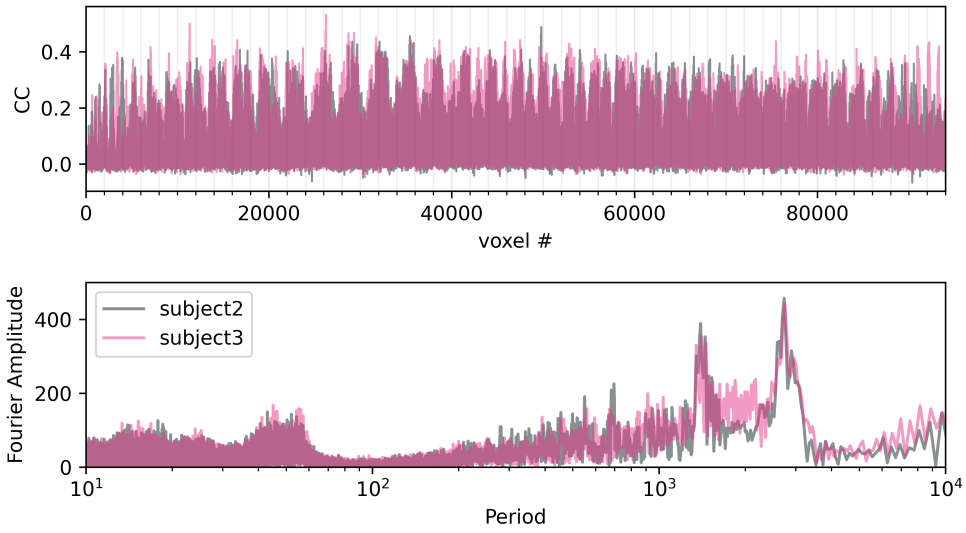


Figure 9: (top) Voxel-wise cross-correlations of the fine-tuned BERT ridge regression model for subject 2 (gray) and subject 3 (pink). (bottom) Fourier transform amplitude of the cross-correlation across the voxels.

5 Interpretation

5.1 First test story: *Leaving Baghdad*

5.1.1 Identifying well-predicted voxels on a test story

We perform a case study on the held-out test story `leavingbaghdad`. We first identify voxels for which the model predicts neural responses well, measured by the Pearson correlation coefficient (CC) between predicted and observed BOLD signals over time. We select the top-performing voxels and focus interpretation on these voxels:

$$\{41267, 31731, 49800, 35377, 35430\}, \quad \text{CCs} = \{0.531, 0.549, 0.554, 0.560, 0.566\}.$$

We only run word-level interpretation methods (SHAP and LIME) on voxels where the predictive performance is reasonably high. This follows the PCS perspective: if a voxel is not predictable by the model (low CC), then any post-hoc explanation of the model's important words is unstable and potentially misleading, since the model is not capturing a reliable signal for that voxel. In other words, interpretability is meaningful only when the underlying prediction is trustworthy; otherwise, the explanation may reflect noise or model artifacts rather than reproducible stimulus-response structure.

5.1.2 SHAP and LIME word influence analysis

For each selected voxel, we compute a SHAP importance score and a LIME importance score over time, identify the most influential TRs, and aggregate the words aligned to those TRs. We then compare the influential words produced by the two methods. Across the selected voxels, SHAP explanations tend to repeatedly surface a small set of highly consistent words (e.g., `didn't, told, where, when, stop`), suggesting that the model attributes prediction changes to a small number of salient narrative moments. In contrast, LIME explanations often produce a more diverse set of content words and scene descriptors (e.g., `america, iraq, baghdad, home, safe, door, screamed`), reflecting sensitivity to locally perturbed inputs and a stronger dependence on specific local contexts. We summarize the overlap between SHAP and LIME words for each voxel below.

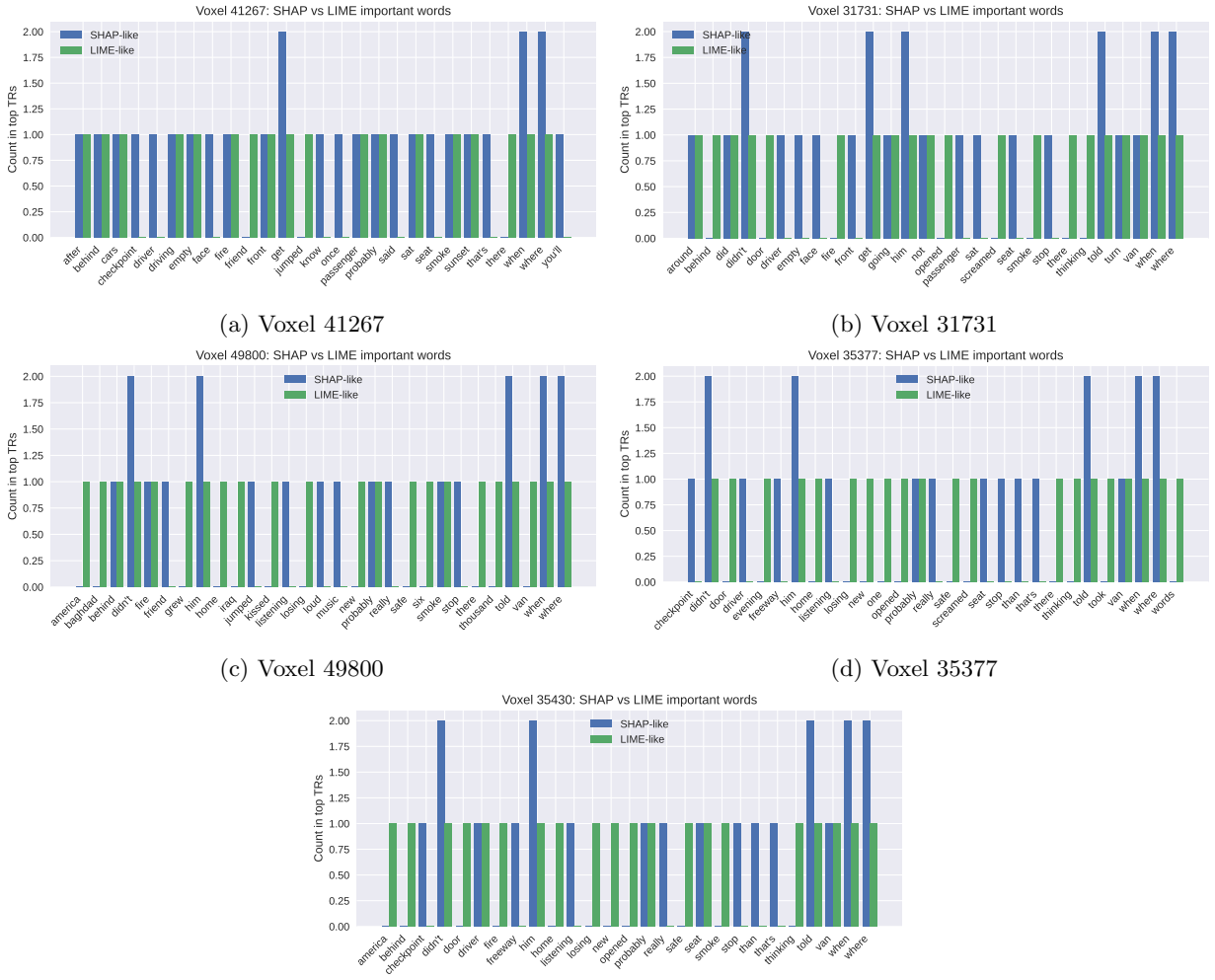
Voxel	CC	SHAP emphasis (examples)	LIME emphasis (examples)
41267	0.531	Repeated function/action words and dialogue markers (e.g., `get, when, where, said`); overlaps include `fire, smoke, behind`.	Scene/action descriptors (e.g., `cars, driving, passenger, sunset, jumped`); overlap with SHAP-like is limited.
31731	0.549	Highly repeated narrative tokens (e.g., `didn't, told, him, get, when, where`); small overlap with LIME-like (`where, van`).	More varied local descriptors (e.g., `turn, around, going, door, fire, smoke`).
49800	0.554	Strong emphasis on repeated narrative anchors (e.g., `didn't, told, him, when, where`) with some danger-related words (`fire, smoke`).	Geopolitical/setting words appear (e.g., `america, iraq, baghdad, home, safe, new`) with limited overlap.
35377	0.560	Recurrent control/scene tokens (e.g., `stop, checkpoint, freeway, driver`) and repeated dialogue words; overlap mainly `van`.	Local events and objects (e.g., `opened, door, screamed, words, thinking`) and setting-related tokens.
35430	0.566	Similar to voxel 35377 with repeated travel/control tokens (e.g., `checkpoint, freeway, stop`) plus `van, driver, seat`.	More diverse contextual words (e.g., `america, home, safe, behind, smoke, fire, door`).

Table 5: Qualitative comparison between SHAP and LIME influential words on `leavingbaghdad`.

Do these words make intuitive sense? Yes. Many influential words reflect coherent narrative elements from the story, including travel/vehicle context (`van, driver, seat`), conflict-related cues (`checkpoint, fire, smoke`), and setting cues (`iraq, baghdad, america, home`). Methodologically, SHAP results are more repetitive and concentrated, while LIME results are more diverse and context-specific, which is consistent with SHAP's global contribution view versus LIME's local surrogate behavior.

5.1.3 Visualization: SHAP vs LIME important words

Figure 10 visualizes the word counts among the top TRs for SHAP and LIME explanations across the selected voxels. The overlap and word types differ across voxels, suggesting that different voxels are sensitive to different semantic aspects of the narrative. Some voxels emphasize travel/control and vehicle-related semantics (e.g., `checkpoint, freeway, driver`), while others exhibit stronger sensitivity to setting and high-level concepts (e.g., `america, iraq, baghdad, home, safe`). This voxel-to-voxel heterogeneity is consistent with distributed semantic representations in language-related cortical areas.



(a) Voxel 41267

(b) Voxel 31731

(c) Voxel 49800

(d) Voxel 35377

(e) Voxel 35430

Figure 10: SHAP vs LIME influential words (aggregated counts among top TRs) for the test story `leavingbaghdad`.

5.2 Second test story: *Wild Women and Dancing Queens*

5.2.1 Identifying well-predicted voxels on a test story

We conduct a focused interpretation case study on the held-out test story `wildwomenanddancingqueens`. Similarly, we restrict interpretation to these voxels:

$$\{4031, 3908, 49760, 54062, 49800\}, \quad \text{CCs} = \{0.575, 0.579, 0.604, 0.611, 0.701\}.$$

5.2.2 SHAP and LIME influential words

A global pattern is that SHAP tends to emphasize highly repeated discourse markers and function-like tokens in this story, especially `we're`, `i'm`, `it's`, and `like`. These tokens appear repeatedly among the top TRs across all selected voxels (and are particularly dominant for voxel 49800 with CC ≈ 0.701), suggesting that the model is sensitive to temporally salient segments characterized by a consistent conversational style or speaker-driven phrasing. In contrast, LIME includes these high-frequency tokens as well, but more often surfaces additional locally specific content words (e.g., `bra`, `underwear`, `camp`, `woods`, `worrying`), consistent with LIME's local perturbation mechanism. We summarize the overlap between SHAP and LIME for each voxel based on the aggregated top words.

Voxel	CC	SHAP (examples)	LIME (examples)
4031	0.575	Strong emphasis on repeated discourse tokens: `I'm` (3), `we're` (3), `it's` (2), `like` (2), plus `cool/wild/super`.	Includes `I'm` (2) and `like`, but surfaces more local content: `bra, underwear, people, just, god`.
3908	0.579	Similar pattern: `I'm` (3), `we're` (3), `about` (2), `it's` (2), `like` (2), plus `staff/night/talking`.	Again includes discourse tokens but adds local items: `bra, underwear, mom, promised, fun`.
49760	0.604	Heavily concentrates on `we're` (4) and `I'm` (3), with some numerical/time terms: `one, day, twenty, fourteen`.	Retains `we're` (3) but adds context words: `camp, worrying, pro, women, know`.
54062	0.611	Mix of discourse tokens and story tone: `I'm` (3), `we're` (3), `it's/about/like`, `wild`.	Keeps `we're, I'm` but introduces setting/action words: `woods, making, standing, chaos`.
49800	0.701	Most predictive voxel; SHAP highlights stable anchors: `we're` (4), `I'm` (3), `like` (2), plus `one/day/twenty/fourteen`.	Shares several anchors (`I'm, like, one/day/twenty`), and adds question/knowledge terms: `know, what, think, camp`.

Table 6: Qualitative comparison of influential words identified by SHAP and LIME on `wildwomenanddancingqueens`. Counts shown in parentheses indicate repeated appearances among aggregated top-TR words.

5.2.3 Visualization: SHAP vs LIME important words

Figure 11 visualizes aggregated word counts among the top TRs for SHAP and LIME across the selected voxels. Overall, the influential words make sense for this particular story: the narrative appears conversational and style-heavy, and the model's strongest signals are driven by repeated discourse markers (e.g., `I'm`, `we're`, `it's`, `like`). Compared to SHAP, LIME more often surfaces concrete local content (e.g., clothing items such as `bra/underwear` and setting cues such as `camp/woods`), suggesting that LIME is more sensitive to localized context while SHAP is more consistent across time. Differences across voxels indicate heterogeneity: some voxels (e.g., 4031/3908) are dominated by discourse/style tokens, while others (e.g., 49760/54062/49800) additionally reflect episodic or setting-specific content.

6 Stability Analysis

To make sure our results hold true or and stay consistent under reasonable variations of the analysis pipeline, we perform a stability analysis following the PCS framework [8]. In the context of high-dimensional fMRI encoding models, this helps ensure that our findings reflect robust signal rather than artifacts introduced by specific judgement calls.

6.1 Stability to Data Cleaning Decisions

As a first stability study, we evaluated whether our data-cleaning choices (primarily filtering out typos and special-case tokens in the text, see Section 2.4) meaningfully affect the voxel-wise prediction performance. Because only a small number of tokens ($\ll 1\%$ of the corpus) were affected by cleaning operations, we expected the cleaned and uncleaned versions of each embedding type to yield nearly identical encoding performance.

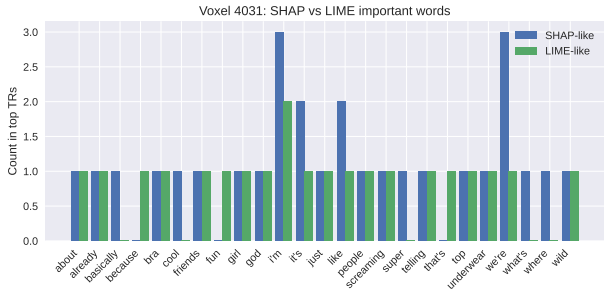
Figure 12 confirms this expectation. For each embedding family, we compared the 95th percentile of voxel-wise correlations between the cleaned and uncleaned pipelines and observed only negligible differences ($\Delta \leq 0.002$) for word2vec. This indicates that our results are stable with respect to the text-cleaning choices.

6.2 Stability to Fine-tuned BERT Window Size

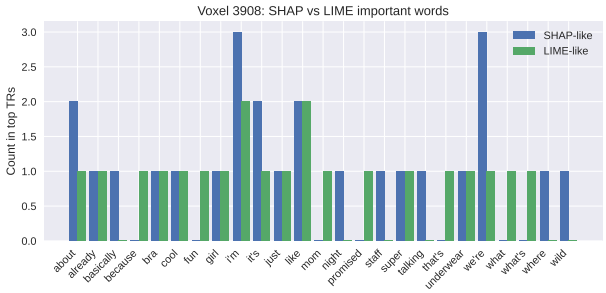
In a second, conceptually deeper stability study we investigated the effect of altering the window size used when preparing training chunks for the finetuned BERT model as the best performing model. Unlike the

data-cleaning step, this perturbation affects a core modeling decision that directly influences semantic context length, token–sentence alignment, and the structure of the embedding space.

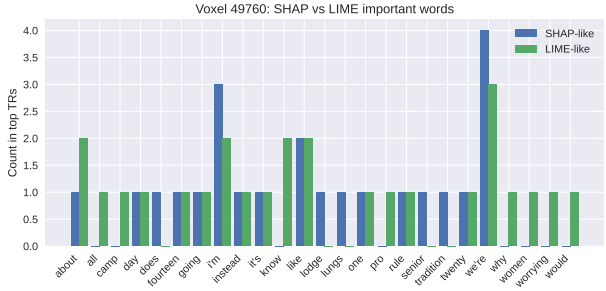
We compared window sizes of 128 and 256 tokens for both the pretrained and finetuned models and observed no major differences. This suggests that a window size of 128 tokens is already sufficient to capture the relevant semantic context for our task.



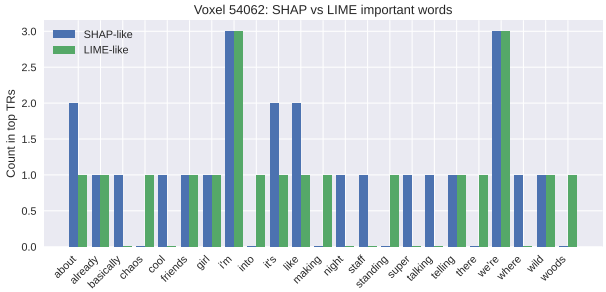
(a) Voxel 4031 (CC=0.575)



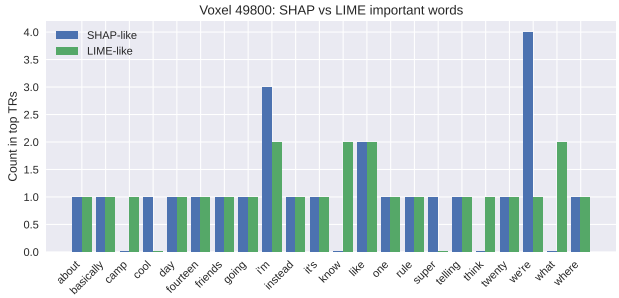
(b) Voxel 3908 (CC=0.579)



(c) Voxel 49760 (CC=0.604)



(d) Voxel 54062 (CC=0.611)



(e) Voxel 49800 (CC=0.701)

Figure 11: SHAP vs LIME influential words (aggregated counts among top TRs) for the test story `wildwomenanddancingqueens`.

7 Conclusion

In this report, we analyzed fMRI (BOLD) responses from two subjects while they listened to 109 different stories, and we developed models to predict the resulting BOLD activity.

We first cleaned the story transcripts through visual inspection, assisted by a standard English dictionary. In this process, we corrected 41 typos and removed 11 non-words. Our exploratory data analysis showed that many words appear repeatedly, and that the most frequent words may be less informative for predicting BOLD responses. This observation suggests that the embedding model should avoid overfitting to high-frequency function words and instead emphasize contextual information when representing each word.

For word embedding development, we evaluated four approaches: Word2Vec, GloVe, a pre-trained BERT

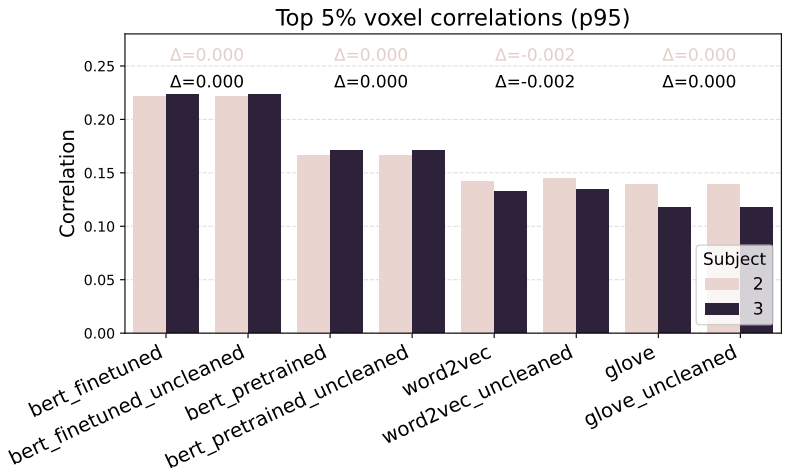


Figure 12: Comparison of the top 5% voxel correlations (95th percentile) for cleaned and uncleaned versions.

model, and a BERT model fine-tuned on the provided dataset. Hyperparameters were selected to emphasize contextual meaning and to mitigate overfitting to frequent words. All four approaches produced reasonable vector representations; however, from a contextual perspective, the BERT-based embeddings appeared to perform better than Word2Vec or GloVe.

We then implemented ridge-regression models to predict the BOLD responses of both subjects. We used an 80/20 train-test split. To handle the prohibitively large design matrices created by high-dimensional embeddings, we implemented a two-pass streaming algorithm. We found that fine-tuned BERT achieved the best performance among the four embeddings, yielding the lowest MSE and highest correlations. The voxel-wise correlation maps also exhibited consistent periodic patterns across both subjects, suggesting that the learned predictive patterns are robust across individuals.

We assessed the importance of individual words for predicting BOLD responses using SHAP and LIME on two stories. By examining the top five most relevant voxels for each story, we found that the influential words differ across stories but still reflect coherent narrative elements specific to each story. For `leavingbagdad`, words related to travel/vehicles, conflict-related cues, and scene-setting cues were identified as important. In contrast, for `wildwomenanddancingqueens`, SHAP tended to emphasize highly repeated discourse markers, whereas LIME highlighted more localized, context-specific words, showing the large variation across different methods.

In addition, to assess the robustness of our results to reasonable variations in the analysis pipeline, we compared regression models trained with and without the transcript-cleaning step. We found that the cleaned and uncleaned pipelines yielded negligible differences in voxel-wise correlations, indicating that our results are stable with respect to the data-cleaning choices.

References

[1] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, 2019.

[2] Bryan A Garner. *Garner's modern English usage*. Oxford University Press, 2016.

[3] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021.

[4] Shailee Jain and Alexander Huth. Incorporating context into language encoding models for fmri. *Advances in neural information processing systems*, 31, 2018.

[5] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient Estimation of Word Representations in Vector Space. In *Proceedings of Workshop at ICLR*, pages 1–12. arXiv, 2013.

[6] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. Glove: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543. Association for Computational Linguistics, 2014.

[7] Wikipedia contributors. Receptive aphasia — Wikipedia, the free encyclopedia, 2025. [Online; accessed 10-December-2025].

[8] Bin Yu and Rebecca Barter. *Veridical Data Science*. MIT Press, Cambridge, MA, 2023.

A Academic honesty Statement

We affirm that all intellectual contributions in this report are the original work of our team. We have cited all sources we used and acknowledged any collaborators with whom we discussed the lab. We only used LLMs to the extent allowed by the lab instructions and used them only to assist us, never to replace our own critical thinking.

We view our research integrity not just as a compliance requirement, but also as a contribution to the collective knowledge base. That is, we believe that academic research honesty is necessary to ensure that, as a community of researchers and, more generally, as a society, we can collectively solve significant and important problems. This requires that we can trust the reliability of the work we build upon. And in practice, our methods must be transparent and our results reproducible.

A.1 Collaborators

We talked to Abdullah Ateyeh and Atiila Kharobo about some ideas related to the project.

A.2 LLM Usage

Coding

- We referenced LLMs to verify library syntax (e.g., pandas, scikit-learn, and matplotlib) and to optimize/improve code snippets throughout the development process.
- One common way we used LLMs was to explain to us how to create the figures using matplotlib and improve our code snippets for the figures. We critically reviewed all outputs from LLMs and rewrote/retyped parts of the code LLMs provided us with, to ensure a thorough understanding of its underlying logic. We did not include any code that we have not verified, and we always first thought ourselves about how and experiment should work or how a figure should look like before consulting an LLM.
- Finally, we used LLMs to generate code comments to make it easier to understand for reviewers.

Writing

- We utilized LLMs to polish our write-up, specifically in correcting spelling, refining grammar, and suggesting more suitable word use to improve the overall readability of the report across all sections.
- We also consulted LLMs to troubleshoot LaTeX formatting issues on Overleaf, specifically to identify necessary packages and resolve compilation errors to achieve a better layout.